

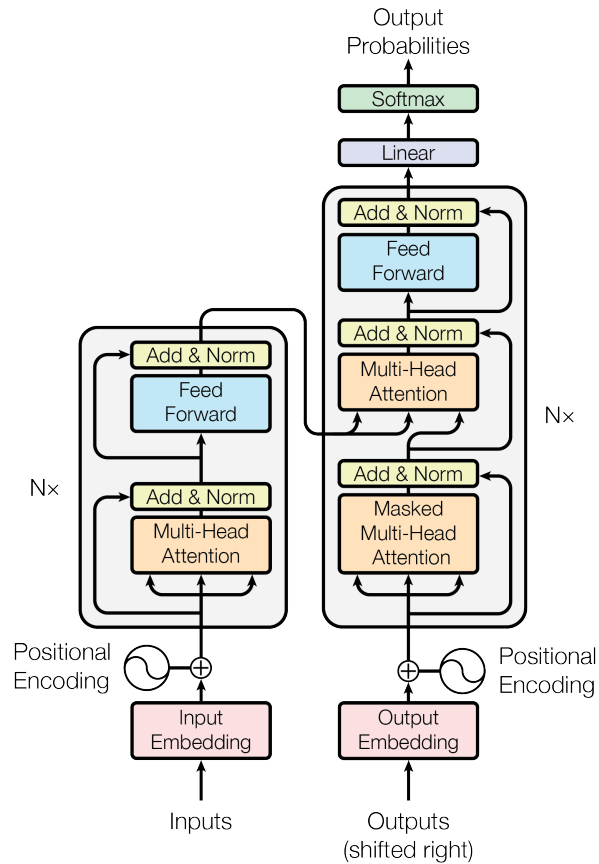


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

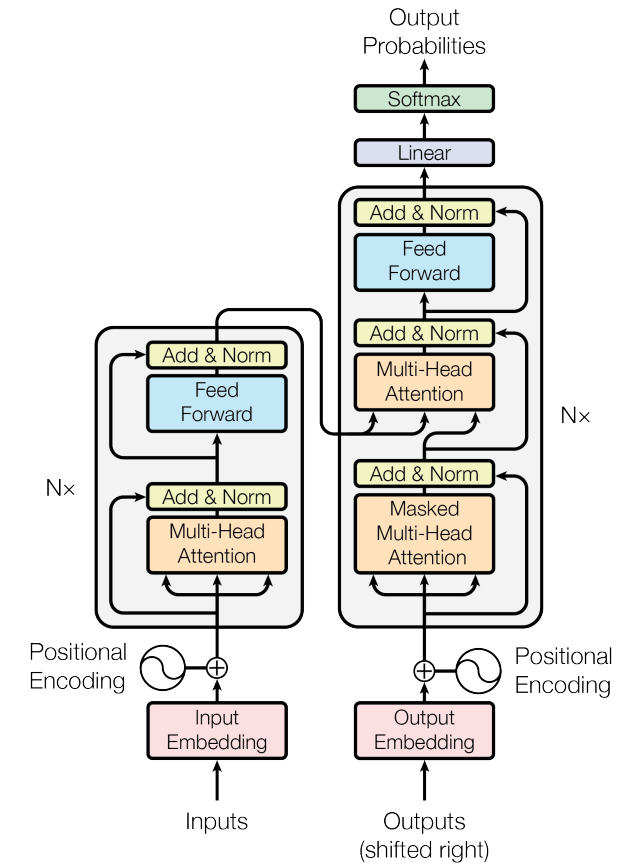


圖 1：Transformer - 模型架構。

Transformer 遵循此整體架構，使用堆疊自注意力與逐點全連接層，分別如圖 1 的左半部和右半部所示，用於編碼器和解碼器。

3.1 編碼器和解碼器堆疊

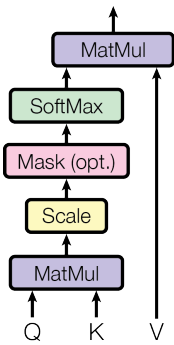
編碼器：編碼器由 $N = 6$ 個相同層的堆疊組成。每個層有兩個子層。第一個是多頭自注意力機制，第二個是簡單的逐點全連接前饋網路。我們在每個子層周圍採用殘差連接 [11]，後接層歸一化 [1]。也就是說，每個子層的輸出是 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是子層本身實現的函數。為便於這些殘差連接，模型中的所有子層以及嵌入層都產生維度為 $d_{\text{model}} = 512$ 的輸出。

解碼器：解碼器也由 $N = 6$ 個相同層的堆疊組成。除了每個編碼器層中的兩個子層外，解碼器還插入第三個子層，該子層對編碼器堆疊的輸出執行多头注意力。與編碼器類似，我們在每個子層周圍採用殘差連接，後接層歸一化。我們還修改了解碼器堆疊中的自注意力子層，以防止位置關注後續位置。這種掩碼，加上輸出嵌入偏移一個位置的事實，確保位置 i 的預測只能依賴於位置小於 i 的已知輸出。

3.2 注意力

注意力函數可以描述為將查詢和一組鍵值對映射到輸出，其中查詢、鍵、值和輸出都是向量。輸出被計算為加權和

Scaled Dot-Product Attention



Multi-Head Attention

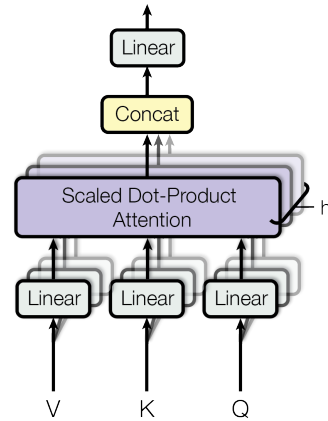


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

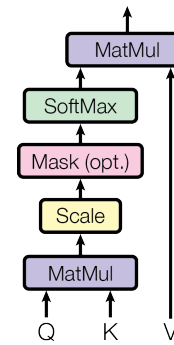
While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .

縮放點積注意力



多头注意力

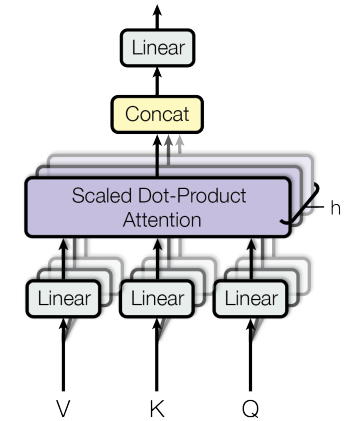


圖2：（左）縮放點積注意力。（右）多头注意力由幾個並行運行的注意力層組成。

值的加權和，其中分配給每個值的權重是通過查詢與對應鍵的兼容性函數計算得出的。

3.2.1 縮放點積注意力

我們將我們特定的注意力稱為「縮放點積注意力」（圖2）。輸入由維度為 d_k 的查詢和鍵，以及維度為 d_v 的值組成。我們計算查詢與所有鍵的點積，將每個點積除以 $\sqrt{d_k}$ ，並應用 softmax 函數來獲取值的權重。

在實際應用中，我們同時計算一組查詢的注意力函數，將它們打包成一個矩陣 Q 。鍵和值也同樣打包成矩陣 K 和 V 。我們計算輸出矩陣如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴舉例說明點積為何會變大，假設 q and k 的分量是均數為 0、變異數為 1 的獨立隨機變數。那麼它們的點積 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 則具有均數 0 和變異數 d_k 。